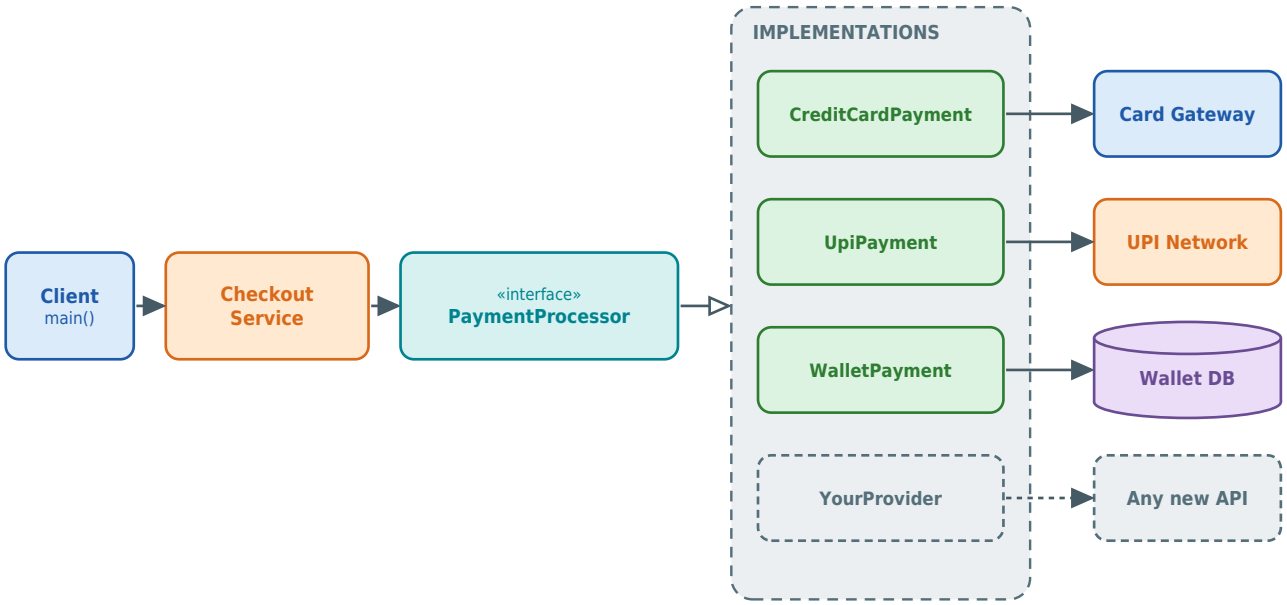
THE MASTERING SYSTEM DESIGN SERIES

Mastering System Design

Low-Level Design (LLD) in Java

A complete, diagram-first guide to object-oriented design in Java: from classes and interfaces to SOLID, all 23 design patterns, concurrency, race conditions, testing and the 60-minute LLD interview.



INSIDE THIS BOOK

Java 21 · OOP · UML · SOLID · Patterns · Concurrency · Race Conditions · Testing · Interview

Edition 1 · Java 21 · 14 Parts · 78 Chapters

About This Book

Who this book is for

This book teaches **low-level design (LLD)** the way a senior Java engineer practises it: by designing real components, compiling them, running them and reviewing the result. It is written for three kinds of reader.

Reader	What you get	Suggested path
Interview candidate	A repeatable 60-minute method, a fully worked example and the follow-up questions interviewers ask	Parts I, V, VI, IX, XII, XIII
Working backend engineer	Vocabulary and techniques for design reviews, refactoring and concurrency bugs	Parts V, VI, VII, VIII, IX, XII
Student or career switcher	A complete path from Java fundamentals to production-grade design	Read Parts I to XIII in order

Every claim about behaviour is backed by a program you can run. Where a topic needs a library that is not part of the JDK (JUnit 5, Mockito, Spring Boot, a JDBC driver), the book says so, shows the idiomatic code labelled as *illustrative*, and also gives a runnable plain-Java equivalent so the lesson never depends on a download.

How the book is organised

The book has **14 parts and 78 chapters**. Each part has its own accent colour, which you will see in the running header, the chapter banner, section headings, tables and listing borders, so you always know which part you are in.

Parts	Theme	What it builds
I to II	Foundations and Java	Vocabulary, workflow, language features used in design
III to V	OOP, UML and principles	Encapsulation to dependency injection, diagrams, SOLID, GRASP, smells, refactoring
VI to VII	Patterns	All 23 GoF patterns, then architecture, DDD, messaging and resilience patterns
VIII to IX	Concurrency	Threads, executors, memory model, locks, races, deadlocks, distributed consistency
X to XI	Data and systems	Data structures, in-memory components, APIs, persistence, security, observability
XII to XIII	Quality and interview	Testing, mocking, code quality, and the 60-minute LLD interview framework
XIV	LLD problems	15 classic interview problems: requirements as points, one UML class diagram, Java code and output

Every chapter opens with an *In this chapter* box, uses numbered sections that appear in the table of contents, and ends with a short list of interview follow-ups or key takeaways. Pattern chapters follow one fixed template: intent, motivating problem, structure and roles, Java code, execution output, sequence flow, benefits, limitations, thread-safety, alternatives, production example and interview follow-ups.

Table of Contents

PART I — FOUNDATIONS OF LOW-LEVEL DESIGN 19

1. Introduction to Low-Level Design 19
- 1.1 What is Low-Level Design? 19
- 1.2 Anatomy of a Good Design 22
- 1.3 Understanding Design Trade-offs 27

2. Anatomy of an LLD Interview 28
- 2.1 What Interviewers Evaluate 28
- 2.2 How a Round Flows 31
- 2.3 Patterns, Concurrency, Errors and Trade-offs 32
- 2.4 Mistakes and Communication 35

3. From Requirements to Object-Oriented Design 37
- 3.1 From Words to Classes 37
- 3.2 Ownership, Collaboration and Lifecycle 39
- 3.3 Designing Incrementally 39
- 3.4 Practical Exercise: Library Management 41

PART II — JAVA FUNDAMENTALS FOR LLD 50

4. Java Language Essentials 50
- 4.1 Java Fundamentals 50
- 4.2 Java Type System 58

5. Classes, Objects and Memory 63
- 5.1 Classes and Objects 63
- 5.2 Object References and Identity 65
- 5.3 Memory: Stack, Heap and Metaspace 68
- 5.4 Design Implications 74

6. Java Interfaces and Abstract Classes 75
- 6.1 Interfaces 75
- 6.2 Abstract Classes 81
- 6.3 Detailed Comparisons 84
- 6.4 Practical Java Examples 86

7. Java Collections and Generics 101
- 7.1 Generics 101
- 7.2 The Collections Framework 104
- 7.3 Ordering, Iteration and Equality 108
- 7.4 Choosing Structures for LLD 111

8. Modern Java Features for Design 114
- 8.1 Functions as Values 114
- 8.2 Optional, Records and Sealed Types 117
- 8.3 Pattern Matching, Switch and Text Blocks 119
- 8.4 Time, Concurrency Starters and Immutability 120

9. Exceptions, Validation and Resource Management 126
- 9.1 The Exception Model 126
- 9.2 Resources and Error Flow 129
- 9.3 Validation and State 133
- 9.4 Handling Errors Responsibly 136

CHAPTER

21 SOLID Principles — Complete Guide

Five principles for classes that are easy to change, each with poor code, a refactoring and UML

IN THIS CHAPTER

21.1 Single Responsibility Principle (SRP)
21.2 Open/Closed Principle (OCP)
21.3 Liskov Substitution Principle (LSP)
21.4 Interface Segregation Principle (ISP)
21.5 Dependency Inversion Principle (DIP)
21.6 Applying SOLID Together

21.1 Single Responsibility Principle (SRP)

Definition and reason to change

> *A class should have **one, and only one, reason to change**.*

The word to focus on is *reason*, not "thing". A reason to change is an *actor* (a person or team) who can ask for a change: finance changes pricing, operations changes the storage, marketing changes the email. If one class answers to all three, three different groups edit the same file, their changes collide, and a bug introduced for one breaks the others.

Class-level responsibility. Ask of any class: *who would ask me to change this?* If the answer is more than one stakeholder, split it. SRP does not mean "one method per class": a class may have many methods, as long as they all serve the same reason to change (an `Invoice` with `addLine`, `total` and `tax` is one responsibility).

God classes and SRP violations

A **god class** is the extreme case: it knows everything and everyone depends on it. Typical symptoms: the name ends in `Manager`, `Util` or `Service` and the file is 1,000+ lines; unrelated imports (`java.sql`, `javax.mail`, a JSON library) in one file; tests that need a database *and* a mail server just to check a price; and merge conflicts every sprint.

Poor implementation

`BadOrderService.placeOrder` prices the cart, writes a row, and sends a mail. Three reasons to change in one method.

Project layout and commands TERMINAL

```
# project layout
src/com/lld/ch21/srp/
    BadOrderService.java              <- POOR: one class with three reasons to change. Pricin...
    OrderParts.java                   <- The small, single-purpose collaborators that replace...
    OrderService.java                 <- REFACTORED: only coordinates the steps; every decisi...
    SrpDemo.java                      <- main class: Same behaviour before and after; only th
# compile every source of this example into the out/ folder, then run the main class
$ javac -d out src/com/lld/ch21/srp/*.java
$ java -cp out com.lld.ch21.srp.SrpDemo
```

UML and roles

UML: Builder

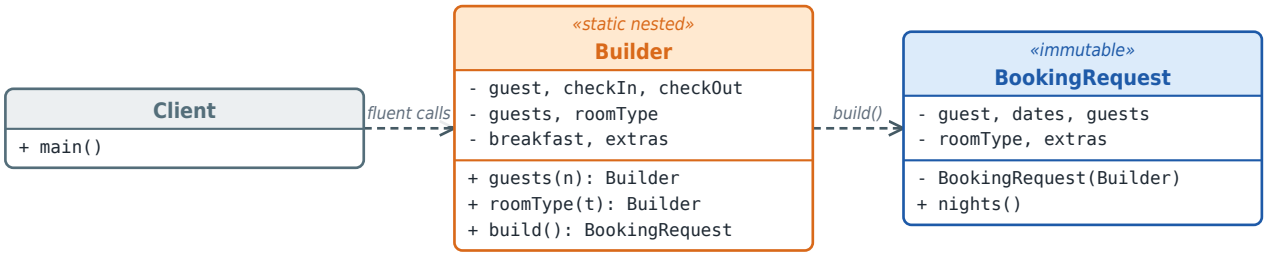


Figure 27.9 — The builder collects values; build() validates and creates the immutable product

Role	Responsibility	In the example
Product	the complex, ideally immutable, object	`BookingRequest`
Builder	accumulates parameters through fluent methods; `build()` validates and creates the product	`BookingRequest.Builder`
Director (optional)	runs a fixed sequence of builder steps for common recipes	a method like `weekendStay(...)`
Client	calls the builder	any caller

Java code and execution output

Required values (`guest`, `checkIn`, `checkOut`) go into the builder's factory method; optional ones are fluent setters with defaults. **All validation happens once, in `build()`,** over the complete set of values, so an invalid or half-built request cannot exist. The product has a private constructor and only `final` fields: it is **immutable**.

Project layout and commands — TERMINAL

```
# project layout
src/com/lld/ch27/builder/
    BookingRequest.java                <- The PRODUCT: an immutable booking request with 3 req...
    BuilderDemo.java                   <- main class: The telescoping-constructor problem vers
# compile every source of this example into the out/ folder, then run the main class
$ javac -d out src/com/lld/ch27/builder/*.java
$ java -cp out com.lld.ch27.builder.BuilderDemo
```

src/com/lld/ch27/builder/BookingRequest.java — JAVA

```java
package com.lld.ch27.builder;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * The PRODUCT: an immutable booking request with 3 required and 4 optional fields.
 * Only the Builder can create one, so an invalid or half-built request cannot exist.
 */
public final class BookingRequest {

    private final String guest;
    private final LocalDate checkIn;
    private final LocalDate checkOut;
    private final int guests;
    private final String roomType;
    private final boolean breakfast;
    private final List<String> extras;

    private BookingRequest(Builder b) {
```

src/com/lld/ch27/builder/BookingRequest.java (continued) JAVA

```
        this.guest = b.guest;
        this.checkIn = b.checkIn;
        this.checkOut = b.checkOut;
        this.guests = b.guests;
        this.roomType = b.roomType;
        this.breakfast = b.breakfast;
        this.extras = Collections.unmodifiableList(new ArrayList<>(b.extras));
    }

    public int nights() {
        return (int) java.time.temporal.ChronoUnit.DAYS.between(checkIn, checkOut);
    }

    @Override
    public String toString() {
        return String.format(
                "%s %s..%s (%dn) %dg %s bf=%b extras=%s",
                guest, checkIn, checkOut, nights(), guests, roomType, breakfast, extras);
    }

    public static Builder builder(String guest, LocalDate checkIn, LocalDate checkOut) {
        return new Builder(guest, checkIn, checkOut);
    }

    /** The BUILDER: required values via the constructor, optional ones via fluent setters. */
    public static final class Builder {
        private final String guest;
        private final LocalDate checkIn;
        private final LocalDate checkOut;
        private int guests = 1;
        private String roomType = "standard";
        private boolean breakfast;
        private final List<String> extras = new ArrayList<>();

        private Builder(String guest, LocalDate checkIn, LocalDate checkOut) {
            this.guest = guest;
            this.checkIn = checkIn;
            this.checkOut = checkOut;
        }

        public Builder guests(int n) {
            this.guests = n;
            return this;
        }

        public Builder roomType(String t) {
            this.roomType = t;
            return this;
        }

        public Builder breakfast(boolean b) {
            this.breakfast = b;
            return this;
        }

        public Builder extra(String e) {
            this.extras.add(e);
            return this;
        }

        /** Validation happens ONCE, here, over the complete set of values. */
        public BookingRequest build() {
            if (guest == null || guest.isBlank()) {
                throw new IllegalStateException("guest name is required");
            }
            if (!checkOut.isAfter(checkIn)) {
                throw new IllegalStateException("check-out must be after check-in");
            }
            if (guests < 1 || guests > 4) {
```

64.3 UML Class Diagram

Catalogue and seats

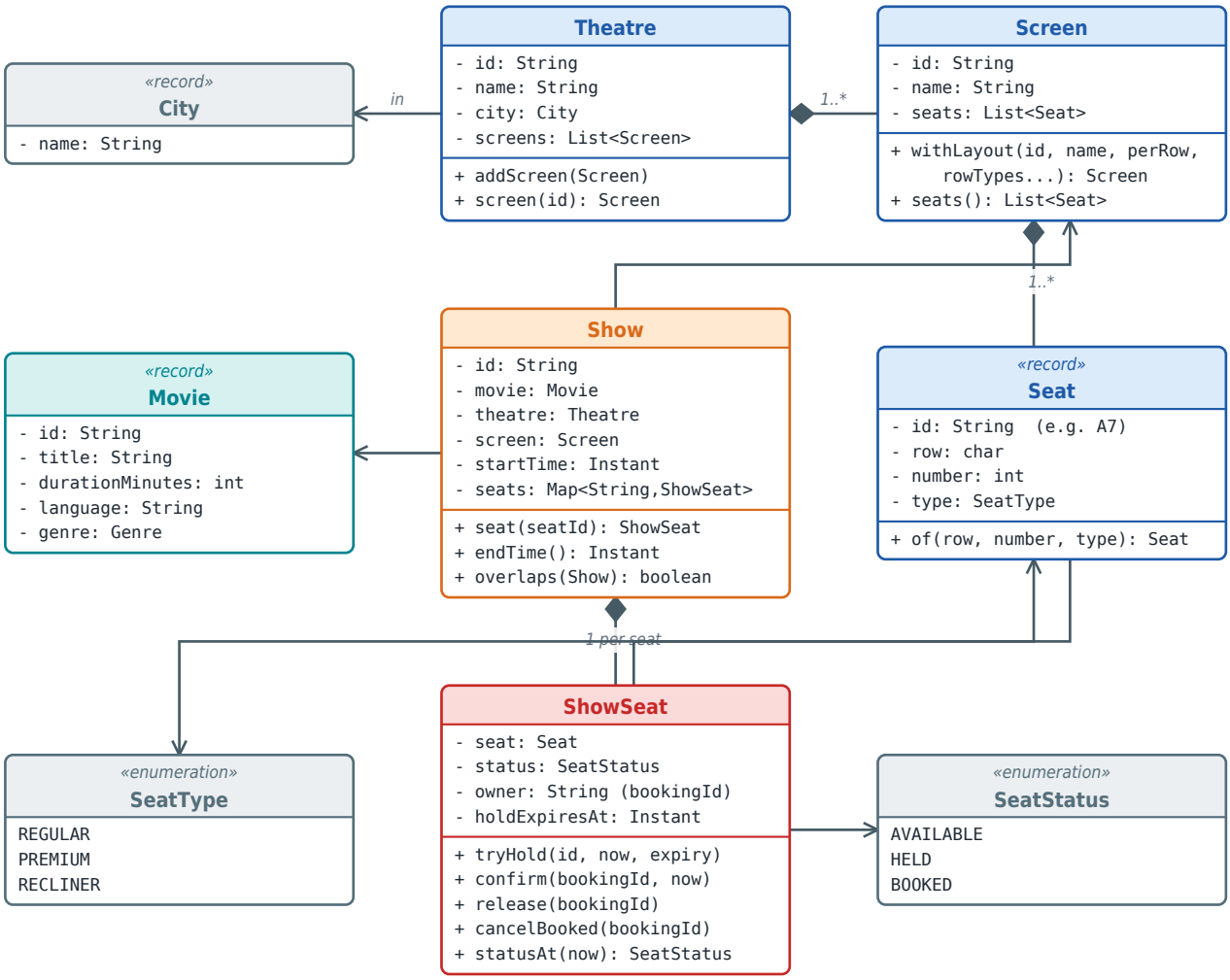


Figure 64.1 — A Seat is physical; ShowSeat is that seat for one show and carries the lock